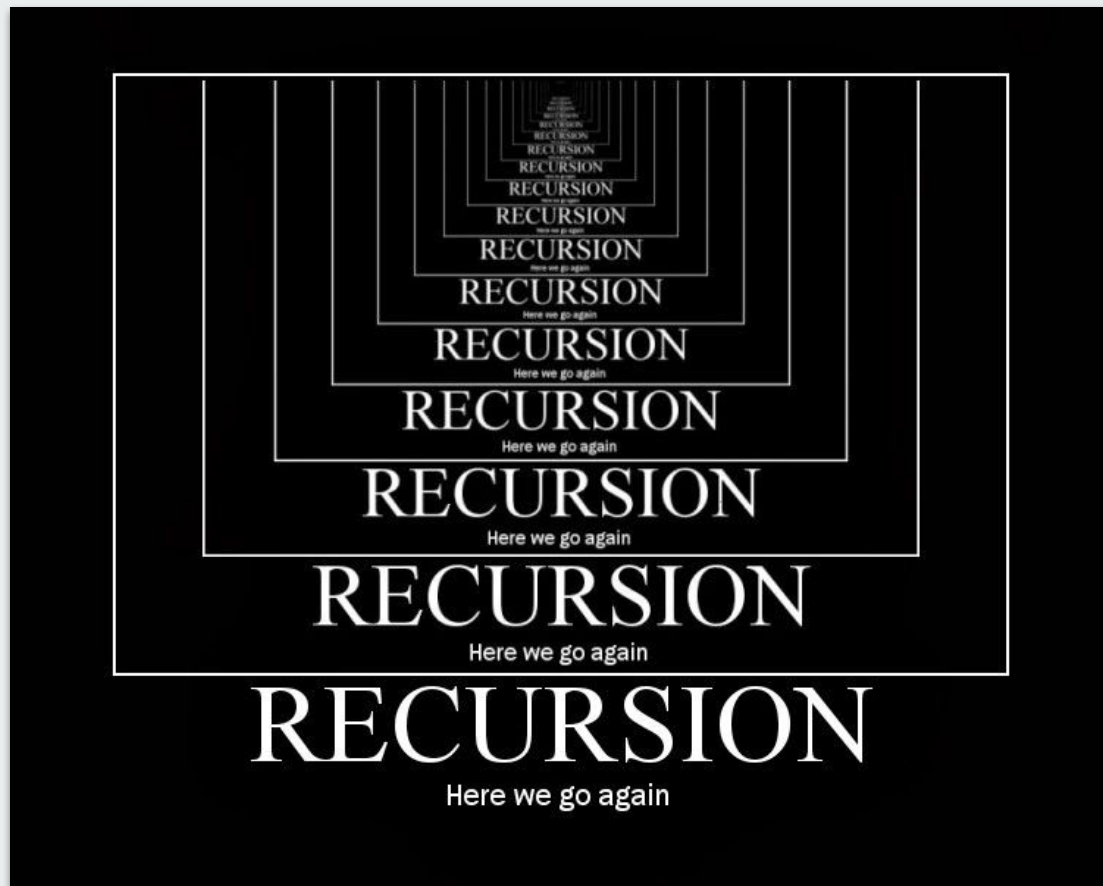




# Recursão

André Rosa  
CTeSP Desenvolvimento  
Software  
Nº2024520  
1ºB





# Recursão?

**Recursividade** (ou recursão) é o processo de repetição de um objeto similar ao que já foi mostrado. Processo pelo qual passa um certo **procedimento** quando um dos passos do **procedimento** em questão envolve a repetição completa deste mesmo **procedimento**.

**Procedimento** é um conjunto de instruções ou passos que devem ser seguidos baseados num conjunto de regras

Um procedimento é **recursivo** quando um dos seus passos consiste na chamada de uma nova execução do procedimento

A **execução de um procedimento** é seguir essas regras a executar as instruções.



# Recursão vs Repetição

**Recursão** é **auto-referencial** - cada passo contém uma versão mais pequena do todo.

**Repetição** é **sequencial** - cada passo segue o anterior sem aninhar-se dentro de si mesmo.

Exemplo:

**Recursão:** História dentro de uma história.

**Repetição:** A mesma história todas as noites.



# Recursão na ciência de computação

Na programação, a recursão pode ser representada de várias maneiras:

- Funções recursivas
- Funções recursivas em cauda
- Recursão indireta
- Recursão aninhada



# Funções Recursivas

```
1 * function fatorial(n) {  
2 *   if (n == 0) {  
3 *     return 1  
4 *   }  
5 *  
6 *   return n * fatorial(n - 1)  
7 * }  
8  
9 fatorial(6); // 720
```

Forma mais simples da recursão na programação.  
Uma função que contém uma invocação de si mesma.  
O exemplo clássico é o **fatorial de N**. Sendo N um número superior a 0, o fatorial de N é a sua consecutiva multiplicação com N-1, até N atingir o valor de 1



## Funções recursivas em cauda

```
1 * function fibonacci(n, a = 0, b = 1) {  
2     if (n === 0) return a;  
3     if (n === 1) return b;  
4     return fibonacci(n - 1, b, a + b);  
5 }  
6  
7 fibonacci(8); // 21
```

Funções recursivas, mas a única diferença é que a chamada recursiva é a última instrução a ser executada.



## Recursão indireta

```
1 * function even(n) {  
2     if (n === 0) return true;  
3     return odd(n - 1);  
4 }  
5  
6 * function odd(n) {  
7     if (n === 0) return false;  
8     return even(n - 1);  
9 }  
10  
11 even(4) // True  
12 odd(4)  // False
```

Funções podem ser recursivas indiretamente, através de outras funções. Exemplo: Função A chama função B, esta que chama a função A.



## Recursão aninhada

```
1 * function ackermann(m, n) {  
2 *   if (m === 0) {  
3 *     return n + 1;  
4 *   } else if (m > 0 && n === 0) {  
5 *     return ackermann(m - 1, 1);  
6 *   } else {  
7 *     return ackermann(m - 1, ackermann(m, n - 1));  
8 *   }  
9 * }  
10  
11 ackermann(1, 2); // 4
```

$$\begin{aligned} A(1, 2) &= A(0, A(1, 1)) \\ &= A(0, A(0, A(1, 0))) \\ &= A(0, A(0, A(0, 1))) \\ &= A(0, A(0, 2)) \\ &= A(0, 3) \\ &= 4. \end{aligned}$$

Uma chamada recursiva pode receber um argumento que inclui outra chamada recursiva. Um exemplo é a **função de Ackermann**.



## Vantagens da recursão

- Clareza e intuitividade
- Redução de linhas de código
- Facilidade em problemas divisíveis
- Flexibilidade em estruturas de dados complexas
- Soluções naturalmente elegantes

## Desvantagens da recursão

- Performance
- Risco de estouro de pilha (**Stack overflow**)
- Dificuldade de Debug
- Consumo de memória
- Complexidade em otimização
- Barreira de compreensão

## Recursão no cotidiano



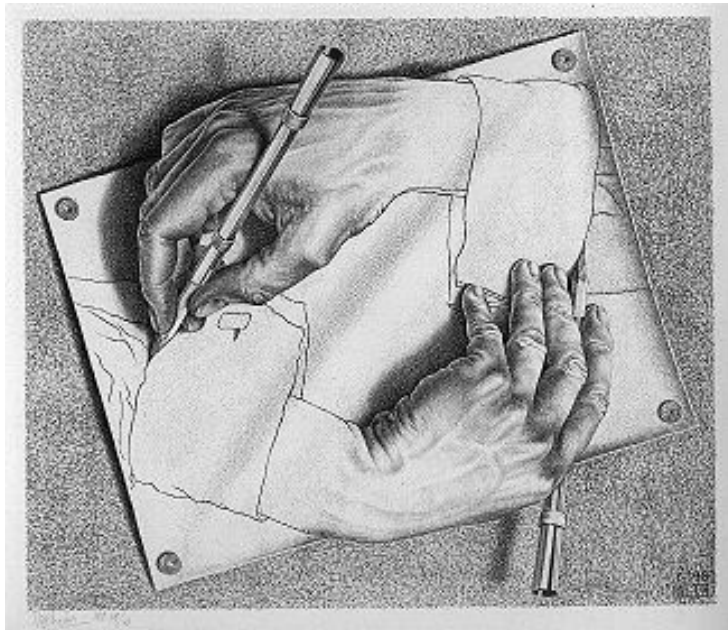
## Matrioshka Dolls (Boneca-russa)



Feitas de madeira, colocadas umas dentro das outras, da maior (exterior) até a menor.

Dentro de uma boneca grande, tem-se uma boneca mais pequena, e dentro desta, outra ainda mais pequena, até se chegar à boneca mais pequena.

## Drawing Hands, M.C Escher



**Maurits Cornelis Escher** (1896 - 1972) foi um artista gráfico holandês conhecido pelas suas xilogravuras (técnica de gravura), litografias (tipo de gravura) e meios-tons, que tendem a representar construções impossíveis. Não tinha conhecimentos matemáticos, mas as suas obras tinham influência recursiva.

Na sua obra **Drawing Hands**, a mão depende de outra mão para que possa existir, e esta, por sua vez, precisa da outra também, criando um ciclo recursivo.



## Recursão na matemática

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n(n-1)! & \text{se } n > 0 \end{cases}$$

$$6! = 6 \times (5!)$$

$$6! = 6 \times 5 \times (4!)$$

$$6! = 6 \times 5 \times 4 \times (3!)$$

$$6! = 6 \times 5 \times 4 \times 3 \times (2!)$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times (1!)$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 \times (0!)$$

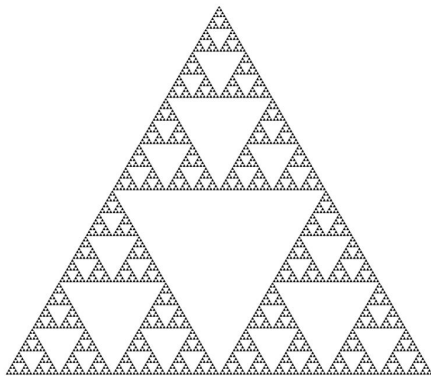
$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 \times 1$$

$$6! = 720.$$

### Definição recursiva do fatorial

De uma forma detalhada, quando:  
 $n = 0$ ;  $0! = 1$   
 $n > 0$ ;  $n! = n(n-1)!$

# Triângulo de Sierpinski

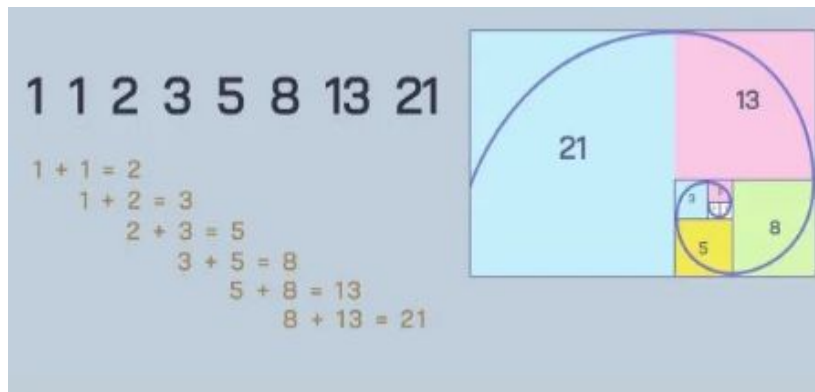


É uma figura geométrica obtida através de um processo recursivo.

É um conjunto autossimilar de um triângulo (normalmente equilátero), que se divide em quatro outros triângulos congruentes entre si e o triângulo original, cujos vértices são os pontos médios do triângulo de origem. O triângulo do meio mantém-se igual, enquanto que outros podem ser divididos infinitamente.



# Sequência de Fibonacci



É uma sequência de números inteiros, começando normalmente por 0 e 1, e cada termo subsequente corresponde à soma dos dois anteriores.

Em termos matemáticos, a sequência é definida recursivamente pela fórmula:  
 $F(n) = F(n-1) + F(n-2)$ , com valores iniciais  $F(0) = 0$  e  $F(1) = 1$ .



## Conclusão

A recursão está em todo o lado, em muitas atividades que realizamos no nosso dia-a-dia. É importante notar também que a recursão pode ser uma ferramenta muito poderosa quando usada corretamente. Difícil de entender, mas elegante.



Além de navegar por estruturas de dados recursivas (por exemplo, um sistema de arquivos), a recursão é às vezes usada em outros lugares. A recursão é uma forma de loop, então tecnicamente pode ser usada sempre que precisarmos fazer algo mais de uma vez.

Aqui está um exemplo esotérico para o lolz: No jogo RTS Warcraft III, você pode ordenar que seus camponeses construam edifícios. Você pode colocar vários edifícios na fila, o que significa que um camponês construirá um edifício, e depois que ele estiver pronto, construirá o próximo, e assim por diante. Os desenvolvedores implementaram isso usando uma chamada recursiva - aparentemente, se o edifício for inconstruível por qualquer motivo, a função `figureOutWhatToDo()` removerá o edifício da fila e, em seguida, chamará a si mesma recursivamente novamente.

Alguém descobriu que você pode abusar disso em um "hack de desconexão" - colocando uma tonelada de edifícios inconstruíveis na fila, você pode fazer com que a recursão transborde. Isso trava todos os jogadores do jogo (então, se você fizer isso quando estiver perdendo, de repente o jogo termina e você não recebe uma derrota em seu registro). Por <https://gamefaqs.gamespot.com/boards/589475-warcraft-iii-the-frozen-throne/52945712>:

---Aspecto técnico: Por que/Como isso trava o jogo?--- Toda vez que você pressiona a tecla F2, o hack emitirá outros 500 pedidos de construção em um local impossível. Agora, se você fizer isso muitas vezes, seu trabalhador terá, por exemplo, 10.000 edifícios na fila. Assim que o trabalhador tentar construir o primeiro edifício da fila, o jogo perceberá que o trabalho não é executável e verificará recursivamente todos os outros na fila até encontrar um executável. Como todos os 10.000 edifícios estão em locais inconstruíveis, a função se chamará 10.000 vezes... o que simplesmente resulta em um estouro de pilha. Cerca de 9.000 edifícios são necessários para preencher a pilha.

Isso também pode ser feito sem nenhum hack, basta segurar shift e colocar 9.000+ edifícios muito, muito rápido no mesmo local. (Você provavelmente ainda precisa de algum tipo de ferramenta de clique para isso) Quando o trabalhador terminar a construção do primeiro edifício, a falha ocorrerá. Também pode ser possível com qualquer unidade e sem usar comandos de construção, ainda não verifiquei isso.

Os desenvolvedores corrigiram isso na [patch 1.24e](#), afirmando apenas que eles "corrigiram uma falha do cliente relacionada a trabalhadores que não conseguiam construir de uma maneira não intencional".